

Proyecto 1. Formulación por Diferencias Finitas:
Método de Diferencia Central para EDOs de
Segundo Orden

Sebastian Matamala, Nicolas Jimenez, Diego Rojas, Arturo Reyes

junio de 2018

1 Descripción del Problema

El programa resuelve numéricamente una ecuación diferencial ordinaria (EDO) de segundo orden de la forma:

$$A x''(t) + B x'(t) + C x(t) = f(t), \quad t \in [0, 1] \quad (1)$$

sujeta a condiciones de frontera tipo Dirichlet:

$$x(0) = x_0 \quad (2)$$

$$x(1) = x_1 \quad (3)$$

donde la función del lado derecho es:

$$f(t) = -t^2 \quad (4)$$

2 Resolución

2.1 Discretización del Dominio

El intervalo $[0, 1]$ se discretiza en $n = \frac{1}{\Delta x}$ subintervalos de tamaño uniforme Δx , generando $n + 1$ nodos $t_i = i \cdot \Delta x$, con $i = 0, 1, \dots, n$.

Los nodos de frontera t_0 y t_n tienen valores fijos x_0 y x_1 . Los nodos interiores $t_1, \dots, t_{n-1}$ son las incógnitas del sistema.

2.2 Aproximación por Diferencias Finitas Centrales

Las derivadas se aproximan usando diferencias finitas centrales:

$$x''(t_i) \approx \frac{x_{i-1} - 2x_i + x_{i+1}}{\Delta x^2} \quad (5)$$

$$x'(t_i) \approx \frac{x_{i-1} - x_{i+1}}{2 \Delta x} \quad (6)$$

Sustituyendo en la EDO se obtiene para cada nodo interior i :

$$A \cdot \frac{x_{i-1} - 2x_i + x_{i+1}}{\Delta x^2} + B \cdot \frac{x_{i-1} - x_{i+1}}{2 \Delta x} + C \cdot x_i = f(t_i) \quad (7)$$

Reordenando en función de los coeficientes de cada nodo:

$$\left(\frac{A}{\Delta x^2} + \frac{B}{2 \Delta x} \right) x_{i-1} + \left(\frac{-2A}{\Delta x^2} + C \right) x_i + \left(\frac{A}{\Delta x^2} - \frac{B}{2 \Delta x} \right) x_{i+1} = f(t_i) \quad (8)$$

$$\begin{bmatrix} \alpha & \beta & 0 \\ \gamma & \alpha & \beta \\ 0 & \gamma & \alpha \end{bmatrix} \begin{bmatrix} T_0 \\ T_1 \\ T_2 \end{bmatrix} = \begin{bmatrix} f(t_1) - \gamma x_0 \\ f(t_2) \\ f(t_3) - \beta x_1 \end{bmatrix}$$

?figurename? 1: Sistema lineal tridiagonal generado por diferencias finitas centrales

2.3 Sistema Lineal Resultante

Para un mallado con $\Delta x = 0.25$ (5 nodos, 3 interiores), el sistema que se forma es:

donde los coeficientes son:

$$\alpha = \frac{-2A}{\Delta x^2} + C \quad (9)$$

$$\beta = \frac{A}{\Delta x^2} - \frac{B}{2\Delta x} \quad (10)$$

$$\gamma = \frac{A}{\Delta x^2} + \frac{B}{2\Delta x} \quad (11)$$

2.4 Ejemplo de Solución

Para los valores $A = 1$, $B = 0$, $C = 0$, $x_0 = 0$, $x_1 = 0$, $\Delta x = 0.25$ (equivalente al problema $x'' = -t^2$), los nodos interiores resultan:

$$\begin{bmatrix} T_0 \\ T_1 \\ T_2 \end{bmatrix} = \begin{bmatrix} -0.01172 \\ -0.01562 \\ -0.01172 \end{bmatrix}$$

?figurename? 2: Vector solución de los nodos interiores

3 Programación

3.1 Descripción General

El sistema se divide en dos scripts:

- **dif_div.py**: genera los polinomios de diferencias finitas y los exporta a un archivo de texto.
- **descomposicion.py**: lee los polinomios, descompone el sistema lineal $Ax = b$ y lo resuelve.

3.2 Función de Discretización y Generación de Polinomios

El siguiente código define las aproximaciones de las derivadas y crea los polinomios simbólicos para cada nodo interior:

```
def ddx(x_a, x, x_p, dx):
    rst = (x_a - 2*x + x_p) / (dx*dx)
    return rst

def d_x(x_a, x_p, dx):
    rst = (x_a - x_p) / (2*dx)
    return rst

for i in range(1, largo-1):
    w[in_nd] = A*ddx(prl[i-1], prl[i], prl[i+1], dx) \
        + B*d_x(prl[i-1], prl[i+1], dx) \
        + C - f_t(stp[i])
    in_nd += 1
```

La función `ddx` aproxima la segunda derivada y `d_x` la primera, ambas con diferencias centrales. El polinomio de cada nodo interior combina ambas aproximaciones ponderadas por A , B y C respectivamente.

Nodo i	x_{i-1}	x_i	x_{i+1}	Coef. $\frac{A}{\Delta x^2}$
1	x_0 (frontera)	T_0	T_1	$A/\Delta x^2$
2	T_0	T_1	T_2	$A/\Delta x^2$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$n-1$	T_{n-3}	T_{n-2}	x_1 (frontera)	$A/\Delta x^2$

?tablename? 1: Estructura de vecindad nodal en la malla 1D

3.3 Mallado Simbólico

Los nodos interiores se representan con variables simbólicas de `sympy` nombradas $T_0, T_1, \dots, T_{n-2}$:

```

prl[0] = x_0
prl[largo-1] = x_1

for i in range(1, largo-1):
    nd = "T" + str(in_nd)
    sy = symbols(nd)
    prl[i] = sy
    in_nd += 1

```

Esto genera el vector de nodos:

$$\mathbf{p} = [x_0, T_0, T_1, \dots, T_{n-2}, x_1]$$

?figurename? 3: Vector de mallado con fronteras fijas y nodos interiores simbólicos

3.4 Función de Descomposición Polinomial

El script `descomposicion.py` lee los polinomios y los separa en la matriz de coeficientes $\mathbf{M}$ y el vector independiente $\mathbf{b}$:

```

def descomponer2(x, y, m):
    if len(x) < 3:
        x[0] = float(x[0]) * m
        l = x[1].split("T")
        l = int(l[1])
        mm[y][l] = x[0]
    else:
        x = float(x) * m
        vct[y] = x

```

La función identifica si el término contiene una variable simbólica T_k (término de coeficiente) o un valor numérico puro (término independiente), llenando la posición correspondiente en la matriz o el vector.

3.5 Resolución del Sistema Lineal

Finalmente, el sistema $\mathbf{M} \mathbf{x} = \mathbf{b}$ se resuelve usando la factorización LU de `numpy`:

```

c = np.linalg.solve(mm, vct)
print("Solucion del sistema")
for i in range(len(c)):
    print(c[i])

```

El resultado entrega los valores de temperatura (o la incógnita modelada) en cada nodo interior, completando la solución con las condiciones de frontera:

$$\mathbf{M} = \begin{bmatrix} c_{0,0} & c_{0,1} & 0 & \cdots & 0 \\ c_{1,0} & c_{1,1} & c_{1,2} & \cdots & 0 \\ 0 & c_{2,1} & c_{2,2} & \cdots & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & \cdots & 0 & c_{n,n-1} & c_{n,n} \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} f(t_1) \\ f(t_2) \\ \vdots \\ f(t_{n-1}) \end{bmatrix}$$

?figurename? 4: Estructura tridiagonal de la matriz del sistema y vector independiente

$$\begin{bmatrix} x(t_0) = x_0 \\ x(t_1) = T_0 \\ x(t_2) = T_1 \\ \vdots \\ x(t_n) = x_1 \end{bmatrix}$$

?figurename? 5: Solución completa del dominio incluyendo condiciones de frontera

3.6 Flujo Completo del Algoritmo

El proceso completo sigue la siguiente secuencia:

1. Ingresar parámetros: x_0 , x_1 , A , B , C , Δx .
2. Generar el mallado de $n + 1$ nodos con variables simbólicas en los nodos interiores.
3. Aplicar la fórmula de diferencias finitas centrales en cada nodo interior para obtener $n - 1$ ecuaciones.
4. Exportar los polinomios a `polinomios.txt`.
5. Leer el archivo, descomponer cada polinomio y construir $\mathbf{M}$ y $\mathbf{b}$.
6. Resolver $\mathbf{M}\mathbf{x} = \mathbf{b}$ con `numpy.linalg.solve`.
7. Imprimir la solución nodal.

3.7 Consideraciones Numéricas

La precisión del método depende directamente del paso Δx :

- El error de truncamiento de la diferencia central es $\mathcal{O}(\Delta x^2)$.

- Valores pequeños de Δx aumentan la precisión pero incrementan el tamaño del sistema.
- La matriz resultante es tridiagonal, lo que permite resolverla eficientemente.
- El programa valida que $0 < \Delta x \leq 1$ antes de proceder.